

MINECRAFT MANAGER

SISTEMA DE ADMINISTRACIÓN DE MUNDOS Y JUGADORES

MANUAL COMPLETO DE DOCUMENTACIÓN TÉCNICA Y GUÍA DE DEFENSA ACADÉMICA

TECNOLOGÍAS PRINCIPALES:

- Backend: Yii 2 REST API (PHP)
- Frontend: Ionic 8 + Angular 20
- Base de Datos: MySQL
- Empaquetado: Capacitor 8

ESTUDIANTE / AUTOR:

Juanito Díaz

FECHA DE PRESENTACIÓN:

Mayo, 2026

TABLA DE CONTENIDOS

1. Introducción y Contexto del Proyecto	Pág. 3
---	--------

.....

2. Arquitectura y Stack Tecnológico	Pág. 3
-------------------------------------	--------

.....

3. Estructura General del Proyecto	Pág. 4
------------------------------------	--------

.....

4. Análisis Comparado con la Rúbrica del Profesor	Pág. 4
---	--------

.....

5. Explicación Detallada del Código por Capas	Pág. 5
---	--------

.....

6. Problemas Técnicos Clave y Soluciones	Pág. 7
--	--------

.....

7. Simulador de Preguntas y Respuestas para la Defensa Oral	Pág. 8
---	--------

.....

8. Estrategia de Exposición y Estructura de la Presentación	Pág. 9
---	--------

.....

9. Conclusiones y Trabajo Futuro	Pág. 10
----------------------------------	---------

.....

10. Glosario Técnico Académico	Pág. 10
--------------------------------	---------

.....

1. INTRODUCCIÓN Y CONTEXTO DEL PROYECTO

El proyecto Minecraft Manager es una aplicación de nivel profesional para la administración de servidores y datos del videojuego Minecraft. Permite centralizar la gestión de las entidades básicas del universo de juego: Mundos (dimensiones y parámetros físicos), Jugadores (usuarios conectados y su progreso), Inventarios (artículos asignados), Items (catálogo de objetos utilizables), Biomas (entornos de generación de terreno) y Mobs (criaturas con sus atributos correspondientes).

Este proyecto resuelve la dificultad de auditar o modificar el estado del servidor directamente sobre archivos binarios o mediante consola in-game. Provee una interfaz móvil intuitiva que interactúa con una base de datos relacional para que cualquier administrador o jugador verifique estadísticas, modifique niveles de experiencia y controle inventarios irregulares de forma remota.

2. ARQUITECTURA Y STACK TECNOLÓGICO

El sistema adopta una arquitectura desacoplada Cliente-Servidor de tipo stateless (sin estado) para garantizar flexibilidad, seguridad y adaptabilidad multi-plataforma:

- Backend RESTful API: Desarrollado en PHP bajo el Framework Yii 2, aprovechando el mapeo relacional de base de datos Active Record.
- Seguridad Stateless: El servidor no maneja cookies ni sesiones en memoria. En su lugar, utiliza tokens Bearer (Authorization: Bearer <token>) validados en base de datos para cada consulta HTTP.
- Frontend Móvil: Implementado en Ionic 8 y Angular 20, permitiendo empaquetar código web como aplicación nativa mediante Capacitor 8.
- Cliente de Conectividad HTTP: Se seleccionó la librería Axios encapsulada dentro de Servicios de Angular que devuelven Observables reactivos.

3. ESTRUCTURA GENERAL DEL PROYECTO

La estructura de archivos organiza limpiamente las capas de la aplicación:

- proyecto/controllers/: Controladores REST encargados de recibir las peticiones HTTP y devolver datos JSON.
- proyecto/models/: Modelos Active Record que manejan las reglas de validación en base de datos y relaciones foráneas.
- proyecto/config/web.php: Configuración central del backend, mapeo de rutas y desactivación del almacenamiento de sesión.
- movil/src/app/services/: Servicios de datos para comunicación HTTP parametrizados con Axios.
- movil/src/app/guard/permiso-guard.ts: Guardia que intercepta la navegación para impedir acceso a secciones según rol.
- movil/src/global.scss: Capa de diseño estilizada que aplica colores y fuentes visuales inspiradas en Minecraft.

4. ANÁLISIS COMPARADO CON LA RÚBRICA DEL PROFESOR

Se realizó un mapeo de las guías de clase para certificar el cumplimiento estricto de las rúbricas establecidas:

Guía / Requisito	Implementación del Proyecto	Estado
01. Listados	Filtro CORS activo en Yii2. Renderizado con *ngFor y Axios.	COMPLETO
02. Detalle	Navegación parametrizada por clave primaria con ActivatedRoute.	COMPLETO
03. Crear	Formularios Reactivos, sesión desactivada (stateless), Token Bearer POST.	COMPLETO
04. Modificar	Paso de ID en modal, patchValue para auto-completar y Axios PUT.	COMPLETO
05. Eliminar	Alerta de confirmación de borrado físico / desvinculación asíncrona.	COMPLETO
08. Paginación	Contenedor Swiper interactivo y llamadas a endpoint /total en API.	COMPLETO
11. Servicios	Llamadas HTTP encapsuladas en Observables inyectables de Angular.	COMPLETO

5. EXPLICACIÓN DETALLADA DEL CÓDIGO POR CAPAS

5.1 Base de Datos y Modelos Active Record (Yii 2)

Los modelos PHP representan las entidades relacionales de la base de datos usando el patrón Active Record de Yii 2:

- **Mundos.php:** Administra la tabla "mundos". Expone relaciones "hasMany" hacia "Biomass" y "JugadoresMundos". Además, redefine la función "beforeDelete" para eliminar en cascada los biomas y las relaciones intermedias con jugadores.
- **Jugadores.php:** Además de representar la tabla, implementa la interfaz IdentityInterface. Esto permite integrar al jugador con el gestor de autenticación nativo mediante su access_token:

```
public static function findIdentityByAccessToken($token, $type = null) {
    return static::findOne(['access_token' => $token]);
}
```

5.2 Controladores REST y Políticas de Acceso (Yii 2)

Los controladores REST heredan de yii\rest\ActiveController. Configuramos la seguridad stateless y las directivas CORS modificando el método behaviors():

```
public function behaviors() {
    $behaviors = parent::behaviors();
    unset($behaviors['authenticator']);
    $behaviors['corsFilter'] = [
        'class' => \yii\filters\Cors::class,
        'cors' => ['Origin' => ['http://localhost:8100', ...]]
    ];
    $behaviors['authenticator'] = [
        'class' => CompositeAuth::class,
        'authMethods' => [HttpBearerAuth::class],
        'except' => ['options']
    ];
    return $behaviors;
}
```

Adicionalmente, redefinimos index para filtrar datos: si el usuario es un "jugador", sólo puede visualizar los mundos e inventarios asociados a su propio identificador.

5.3 Servicios de Comunicación HTTP (Ionic/Angular)

En lugar de usar peticiones Axios directas en las páginas, encapsulamos las llamadas en servicios de Angular para mantener el desacoplamiento de código. Los métodos devuelven un Observable sobre el cual los componentes se suscriben de manera reactiva:

```
listado(extra: string = "", busqueda: string = "): Observable<any> {
  let url = busqueda === "" ? this.url : `${this.url}/buscar/${busqueda}`;
  return new Observable(observer => {
    axios.get(url, { withCredentials: true, headers: this.getHeaders() })
      .then(response => {
        observer.next(response.data);
        observer.complete();
      }).catch(err => observer.error(err));
  });
}
```

5.4 Flujos de Autenticación, Registro y Guards (Ionic/Angular)

El login del usuario envía las credenciales en formato JSON al endpoint /jugadores/login. Si son válidas, el backend devuelve el access_token, el cual es almacenado localmente en el dispositivo. La protección de rutas críticas se realiza mediante un Route Guard (permisoGuard) que intercepta la navegación y verifica si el rol guardado en localStorage cuenta con permisos de acceso registrados en base de datos. De lo contrario, impide la navegación y redirige.

5.5 Formularios Reactivos y Carga de Archivos (Ionic/Angular)

En la creación de jugadores, se maneja una colección dinámica de mundos mediante un FormArray de Angular. Para la carga de imágenes, capturamos el archivo binario del evento change y lo enviamos empaquetado en un objeto FormData con cabeceras multipart/form-data:

```
const formData = new FormData();
formData.append('foto', this.archivoSeleccionado);
await axios.post(this.baseUrl + '/subir-foto/' + id, formData, {
  headers: { 'Authorization': 'Bearer ' + localStorage.getItem('token') }
});
```

6. PROBLEMAS TÉCNICOS CLAVE Y SOLUCIONES

6.1 Desincronización de Relaciones Muchos a Muchos

El Problema: La relación entre jugadores y mundos se gestiona en la tabla intermedia "jugadores_mundos". Al modificar un jugador existente en la aplicación móvil, si deseleccionamos mundos del formulario dinámico (FormArray), las peticiones POST de guardado de la app sólo registraban las nuevas selecciones pero no eliminaban los mundos desvinculados, causando datos huérfanos o duplicidades.

La Solución: Implementar un paso de desvinculación previo que vacía las relaciones antiguas antes de persistir la nueva lista, garantizando la consistencia relacional atómica del modelo.

6.2 Integridad Referencial y Borrado en Cascada en Backend

El Problema: Eliminar un mundo causaba fallos de servidor (Error 500) debido a la existencia de claves foráneas con las tablas de biomas y jugadores. MySQL bloqueaba el borrado físico del registro para prevenir inconsistencias.

La Solución: Sobreescibir el método beforeDelete() en el modelo Mundos.php de Yii 2 para estructurar un borrado lógico ordenado jerárquicamente:

```
public function beforeDelete() {
    if (!parent::beforeDelete()) { return false; }
    // 1. Elimina las relaciones intermedia con jugadores
    JugadoresMundos::deleteAll(['id_mundo' => $this->id]);
    // 2. Borra secuencialmente mobs y biomas asociados
    foreach ($this->biomas as $bioma) {
        Mobs::deleteAll(['id_bioma' => $bioma->id]);
        $bioma->delete();
    }
    return true;
}
```

6.3 CORS e Interceptor Global de Axios

El Problema: Inyectar el token Bearer en cada llamada de Axios duplicaba código y propiciaba fallos de autorización si algún programador olvidaba incluir la cabecera en nuevos desarrollos.

La Solución: Definir un interceptor en AppComponent.ts que captura de forma automática toda petición saliente de Axios e inserta el token Bearer en las cabeceras de autorización de manera centralizada.

6.4 Problema del Buscador con Campos Nulos (CONCAT vs CONCAT_WS)

El Problema: Al realizar búsquedas en los listados (como buscar al jugador "jumo_10" o buscar mobs/mundos con datos nulos), el buscador no devolvía resultados. Esto ocurría porque en SQL la expresión CONCAT(col1, ' ', col2) retorna NULL si alguno de los campos concatenados es NULL. En consecuencia, si un jugador no tenía registrada una fecha de unión o un mundo no tenía semilla/dificultad, la fila completa era excluida de la búsqueda.

La Solución: Cambiamos la función CONCAT por CONCAT_WS(' ', col1, col2, ...) (Concatenate With Separator). CONCAT_WS ignora automáticamente los valores NULL sin invalidar toda la cadena de texto, permitiendo realizar búsquedas correctas sobre cualquier registro.

6.5 Falta del Componente Buscador en Vista de Mundos y Accesibilidad

El Problema: La vista de Mundos carecía del componente `<ion-searchbar>` en el archivo HTML, impidiendo realizar búsquedas en este módulo tanto a administradores como a usuarios estándar. Además, se requería etiquetar el buscador con un atributo `aria-label="serch text"` para el lector de pantalla / scripts de pruebas automatizadas del profesor.

La Solución: Agregamos el componente `<ion-searchbar>` en `mundos-listado.page.html` e inyectamos el atributo `aria-label="serch text"` en todos los buscadores del proyecto.

7. SIMULADOR DE PREGUNTAS Y RESPUESTAS (DEFENSA ORAL)

A continuación se presentan las preguntas técnicas complejas que podría realizar el jurado y las respuestas profesionales estructuradas para el estudiante:

P1: ¿Por qué decidieron utilizar autenticación sin estado (stateless) en lugar de sesiones tradicionales de PHP en Yii 2?

Respuesta: "El cliente móvil es una aplicación desacoplada (SPA) empaquetada para celulares. Las sesiones de PHP tradicionales basadas en cookies de sesión no son viables ni escalables entre dominios diferentes. Al desactivar las sesiones tradicionales (`enableSession = false`) y validar las consultas con tokens Bearer en cada petición HTTP, logramos una arquitectura stateless ideal para móviles, permitiendo al servidor procesar peticiones independientes y reduciendo el consumo de memoria en backend."

P2: ¿Cómo funciona el flujo de validación de permisos y qué seguridad real aporta?

Respuesta: "En la app móvil, el Route Guard (permisoGuard) lee del localStorage las vistas permitidas y restringe la navegación para brindar una experiencia de usuario fluida y coherente. Sin embargo, la seguridad final y robusta se ejecuta en el backend: cada endpoint REST de Yii 2 ejecuta `checkAccess()` antes de persistir datos, comparando el rol de base de datos asociado al `access_token` y arrojando una excepción HTTP 403 (Acceso Denegado) si el usuario no tiene permisos válidos."

P3: Si la base de datos arrojaba un error de Foreign Key Constraint al eliminar un mundo, ¿por qué no desactivar las restricciones de llaves foráneas en MySQL en vez de programar un beforeDelete en PHP?

Respuesta: "Desactivar las llaves foráneas rompería las reglas fundamentales de consistencia relacional de base de datos y provocaría la existencia de registros huérfanos (por ejemplo, biomas sin mundos o mobs apuntando a zonas inexistentes). Al programar `beforeDelete()` en PHP garantizamos que el sistema elimine de forma jerárquica y transaccional los registros dependientes de forma controlada y segura."

P4: ¿Cuál es el beneficio de inyectar las cabeceras HTTP mediante un interceptor global de Axios en lugar de hacerlo en cada petición de los servicios?

Respuesta: "El interceptor global proporciona mantenibilidad y centralización. Garantiza que cualquier petición HTTP saliente cuente de forma transparente con el token de autorización actualizado de localStorage, eliminando la duplicación de código en todos nuestros servicios de Angular y previniendo errores por omisión humana en futuras expansiones."

8. ESTRATEGIA DE EXPOSICIÓN Y ESTRUCTURA DE LA PRESENTACIÓN

Sigue esta secuencia ordenada durante tu tiempo de exposición para transmitir seguridad y dominio técnico:

1. Introducción del Proyecto (2 minutos): Saludo formal, explicar que Minecraft Manager es un administrador centralizado para servidores de juego que separa backend REST de frontend móvil híbrido.
2. Demostración de Funcionalidad y UI (4 minutos): Iniciar sesión como administrador. Mostrar las listas e interactuar con el componente de paginación Swiper y la barra de búsqueda (explicar que el debounce evita saturar al servidor).
3. Demostración de Creación y Seguridad (3 minutos): Registrar un jugador agregando foto y asociando mundos. Explicar el funcionamiento de FormArray y FormData. Luego ingresar con el rol del jugador creado para demostrar cómo la app oculta accesos y cómo el backend filtra automáticamente la información (seguridad stateless).
4. Conclusión y Cierre (1 minuto): Resaltar la robustez y escalabilidad de la arquitectura desacoplada y dar la palabra al jurado para preguntas.

9. CONCLUSIONES Y TRABAJO FUTURO

Conclusiones:

La integración exitosa de Yii 2 REST API y un cliente Ionic/Angular modularizado demuestra que las tecnologías desacopladas son el estándar moderno para el desarrollo de software seguro y multiplataforma. La desactivación de sesiones y la validación por tokens Bearer certifica un diseño backend escalable y moderno.

Posibles Mejoras Futuras:

- Websockets en vivo: Sincronizar en tiempo real las coordenadas de los jugadores activos dentro de los mundos.
- Conexión RCON: Permitir que la app móvil envíe comandos directos a la consola de un servidor oficial de Minecraft.
- Caché Redis: Almacenar temporalmente consultas recurrentes de biomas y mobs en el servidor para reducir latencias.

10. GLOSARIO TÉCNICO ACADÉMICO

- API RESTful: Interfaz de software que utiliza peticiones HTTP para transferir representaciones de datos entre sistemas independientes en formato JSON.
- Active Record: Patrón donde una clase de programación representa una tabla de base de datos y una instancia de dicha clase representa una fila específica.
- Stateless (Sin Estado): Principio arquitectónico donde cada solicitud HTTP del cliente contiene toda la información necesaria para procesarse, sin depender de sesiones almacenadas en memoria del servidor.
- CORS (Cross-Origin Resource Sharing): Mecanismo que utiliza cabeceras HTTP adicionales para permitir que un navegador web obtenga recursos de un origen distinto al de la página actual.
- Route Guard: Elemento de seguridad frontend que intercepta e inspecciona las peticiones de navegación de pantalla, bloqueándolas en caso de no cumplir criterios de autenticación o rol.
- Debounce: Técnica de desarrollo web que pospone la ejecución de un proceso de búsqueda asíncrona hasta que el usuario haya dejado de escribir por un lapso breve, reduciendo la carga del servidor.
- FormArray: Estructura reactiva de Angular para gestionar una cantidad variable de controles del formulario (inputs) de forma dinámica.
- Integridad Referencial: Regla de base de datos que garantiza que cada clave externa apunte a una clave primaria existente, previniendo inconsistencias lógicas en tablas dependientes.